

实验七 数组程序设计（一）一维数组与排序算法

一、 实验目的：

1. 熟练掌握一维数组编写程序；
2. 理解几个常用的排序算法并通过数组编程实现。

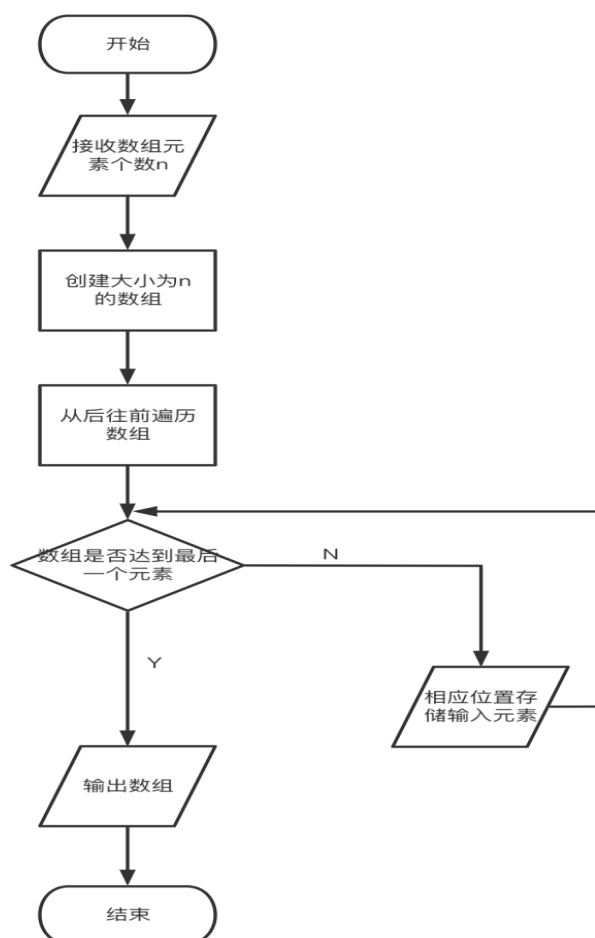
二、 实验要求：

1. 用 C 语言实现源程序的编写；
2. 源程序应书写规范，适当添加注释；
3. 实验课前完成实验内容源程序的初稿；实验课对源程序进行编辑、调试、运行；实验课后尽快完成实验报告，并按规定时间上交实验报告。

三、 实验源代码、结果及流程图：

1. 输入一个正整数 n ($1 < n \leq 10$)，再输入 n 个整数并存入数组，然后将数组中的这 n 个数逆序存放并输出存放结果。试编写相应程序（本题要求在实验报告中画出流程图）。

程序流程图：



```

#include <stdio.h>

//6 1 2 3 4 5 6
int main(int argc, char *argv[]) {
    int n;
    scanf("%d", &n);

    int num[n];
    for(int i = n - 1; i >= 0; --i)
        scanf("%d", &num[i]);

    printf("存放结果如下: \n");
    for(int i = 0; i < n; i++)
        printf("%d ", num[i]);
}

```

测试结果:

```

6 1 2 3 4 5 6
存放结果如下:
6 5 4 3 2 1

```

2. 求一批整数中出现最多的数字。输入一个正整数 n ($1 < n \leq 10$)，再输入 n 个整数，分析每个整数的每一位数字，求出现次数最多的数字。例如输入 3 个整数 1234、2345、3456，其中出现次数最多的数字是 3 和 4，均出现了 3 次。试编写相应程序。

```

#include <stdio.h>

//利用好全局变量
int res[999];
int frequency[999];
int vis[999];
int i = 0;

int getDigit(int num) {
    int tmp;
    while(num) {
        tmp = num % 10; //获取个位数字
        res[i++] = tmp;
        num /= 10;
    }
}

```

```

    }
    return i; //每次对 i 进行更新
}

void getFrequency(int len, int x, int index) {
    int cnt = 0;
    for(int i = 0; i < len; i++) {
        if(vis[i])
            continue;
        if(res[i] == x) {
            cnt++;
            vis[i] = 1;
        }
    }
    frequency[index] = cnt;
}

int getMax(int len) {
    int max = frequency[0];
    for(int i = 1; i < len; i++)
        if(frequency[i] > max)
            max = frequency[i];
    return max;
}

//3 1234 2345 3456
int main(int argc, char *argv[]) {
    int n;
    scanf("%d", &n);

    int num[n];
    for(int i = 0; i < n; ++i)
        scanf("%d", &num[i]);

    int len = 0;
    for(int i = 0; i < n; ++i) //获取每一位数字，存入数组中
        len = getDigit(num[i]);

    for(int i = 0; i < len; ++i) //获取每一位数字出现的频率，数组
        getFrequency(len, res[i], i);

    int max = getMax(len);
    printf("出现最多次数的数字如下:\n");
    for(int i = 0; i < len; ++i) //打印最大频率的数字

```

```
        if(frequency[i] == max)
            printf("%d ", res[i]);
    }
```

测试结果:

3 1234 2345 3456

出现最多次数的数字如下:

4 3

3. 选择排序。输入一个正整数 n ($1 < n \leq 10$)，再输入 n 个整数，将它们从大到小排序后输出。试编写相应程序。

```
#include <stdio.h>

int getMin(int n, int begin, int a[], int *index) {
    int min = a[begin];
    for(int i = begin + 1; i < n; i++)
        if(a[i] < min) {
            min = a[i];
            *index = i;
        }
    if(min == a[begin]) //最小元素就是自身
        *index = begin;
    return min;
}

void dispSolution(int n, int a[]) {
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);
}

void selectSort(int n, int a[]) { //每一位都要遍历到，可以另编一个函数
    //判断数组是否升序了，进行优化
    int index;
    for(int i = 0; i < n; i++) {
        int min = getMin(n, i, a, &index);
        int tmp = a[i];
        a[i] = min;
        a[index] = tmp;
    }
}
```

```

//          dispSolution(n, a);
//          printf("\n");
    }
}

//8 82 74 51 42 36 23 10 20
int main(int argc, char *argv[]) {
    int n;
    scanf("%d", &n);

    int num[n];
    for(int i = 0; i < n; ++i)
        scanf("%d", &num[i]);

    selectSort(n, num);
    printf("选择排序后:\n");
    dispSolution(n, num);
}

```

测试结果:

8 82 74 51 42 36 23 10 20

选择排序后:

10 20 23 36 42 51 74 82

4. 冒泡排序。输入一个正整数 n ($1 < n \leq 10$)，再输入 n 个整数，将它们从小到大排序后输出。试编写相应程序。

```

#include <stdio.h>

void bubbleSort(int a[], int n){ //从小到大升序输出
    int tmp;
    for(int i = n-1; i >= 1; i--){
        int flag = 0;
        for(int j = n - 1; j >= n - i; j--){
            tmp = a[j];
            if(tmp < a[j - 1]){
                flag = 1;    //一轮冒泡中，交换了元素
                a[j] = a[j - 1];
                a[j - 1] = tmp;
            }
        }
        if(flag == 0) break;
    }
}

```

```

        }
    }
    if(flag == 0) //未交换元素，提前结束即可（时间性能得到
优化）
        break;
    }
}

void dispSolution(int n, int a[]) {
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);
}

//8 82 74 51 42 36 23 10 20
int main(int argc, char *argv[]) {
    int n;
    scanf("%d", &n);

    int num[n];
    for(int i = 0; i < n; ++i)
        scanf("%d", &num[i]);

    bubbleSort(num, n);
    printf("冒泡排序后:\n");
    dispSolution(n, num);
}

```

测试结果:

8 82 74 51 42 36 23 10 20

冒泡排序后:

10 20 23 36 42 51 74 82

四、 实验小结:

1. 对于项目的需求，熟练掌握了如何利用一维数组存储所需要的数据，以及如何将实际含义抽象成数组元素，架起了计算机和现实的桥梁。
2. 经过独立的编写选择排序，冒泡排序，理解了常用的排序算法，以及循环的嵌套和数组之间的对应关系。

3. 对于经典的排序算法，在排序过程中，可能已经排好序了，要利用内存空间记录加以优化，达到以空间换时间的效果。

五、实验成绩：